

Multi-Agent Reinforcement Learning for Dynamic Topology Optimization of Mesh Wireless Networks

Wei Sun[✉], Senior Member, IEEE, Qiushuo Lv[✉], Yang Xiao[✉], Fellow, IEEE, Zhi Liu[✉], Senior Member, IEEE, Qingwei Tang, Qiyue Li[✉], Senior Member, IEEE, and Daoming Mu

Abstract—In Mesh Wireless Networks (MWNs), the network coverage is extended by connecting Access Points (APs) in a mesh topology, where transmitting frames by multi-hop routing has to sustain the performances, such as end-to-end (E2E) delay and channel efficiency. Several recent studies have focused on minimizing E2E delay, but these methods are unable to adapt to the dynamic nature of MWNs. Meanwhile, reinforcement-learning-based methods offer better adaptability to dynamics but suffer from the problem of high-dimensional action spaces, leading to slower convergence. In this paper, we propose a multi-agent actor-critic reinforcement learning (MACRL) algorithm to optimize multiple objectives, specifically the minimization of E2E delay and the enhancement of channel efficiency. First, to reduce the action space and speed up the convergence in the dynamical optimization process, a centralized-critic-distributed-actor scheme is proposed. Then, a multi-objective reward balancing method is designed to dynamically balance the MWNs' performances between the E2E delay and the channel efficiency. Finally, the trained MACRL algorithm is deployed in the QaulNet simulator to verify its effectiveness.

Index Terms—Actor-critic, mesh wireless network, reinforcement learning, topology optimization, ad hoc wireless network.

NOMENCLATURE

n	The number of APs, and the number of actors.
T	The maximum training time of the trajectory.
N	The maximum number of the trajectory.
α	The learning rate of actor.
λ	The discount coefficient.
γ	The exponential decay rate.
η	The data packet arrives rate.
τ_k	The k -th episode trajectory.

Manuscript received 23 April 2023; revised 12 November 2023 and 16 January 2024; accepted 26 February 2024. Date of publication 14 March 2024; date of current version 12 September 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 52277087, Grant 62173120, and Grant 52077049; in part by the Natural Science Foundation of Anhui Province under Grant 2108085UD07, Grant 2108085UD11, and Grant 2008085UD04; and in part by the 111 Project under Grant BP0719039. The associate editor coordinating the review of this article and approving it for publication was K. Choi. (Corresponding author: Qiyue Li.)

Wei Sun, Qiushuo Lv, Qingwei Tang, Qiyue Li, and Daoming Mu are with the School of Electrical and Automation Engineering, Hefei University of Technology, Hefei, Anhui 230009, China, and also with Anhui Engineering Technology Research Center of Industrial Automation, Hefei 230009, China (e-mail: liqiyue@mail.ustc.edu.cn).

Yang Xiao is with the Department of Computer Science, The University of Alabama, Tuscaloosa, AL 35487 USA.

Zhi Liu is with Department of Computer and Network Engineering, The University of Electro-Communications, Tokyo 182-8585, Japan.

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TWC.2024.3372694>.

Digital Object Identifier 10.1109/TWC.2024.3372694

ζ	The probability of the AP trying to send frames.
θ_i	The neural network parameters of actor i .
ΔS	The variation in transmission power.
$\forall$	True if all cases are successful.
$\exists$	True if one of the conditions is successful.
$\wedge$	Logical AND.
S_{it}^k	AP i 's state on the k -th trajectory at time t .
S_t^k	The state vector on the k -th trajectory at time t .
a_{it}^k	AP i 's action on the k -th trajectory at time t .
$\mathbf{a}_t^k$	The action vector on the k -th trajectory at time t .
$Q(S_t^k, \mathbf{a}_t^k)$	The state-action value function on the k -th trajectory at time t .
$H(S_t^k)$	The advantage function vector on the k -th trajectory at time t .
$R(S_{it}^k)$	The reward on the k -th trajectory at time t of AP i .
$\mathbf{R}(S_t^k)$	The reward vector on the k -th trajectory at time t .
$V(S_t^k)$	The state value function on the k -th trajectory at time t .
$R_{\theta_i}^k$	The mathematical expectation for cumulative reward from the trajectory τ_k of AP i .
$T(S_{it}^k)$	The E2E delay on the k -th trajectory of AP i at time t .
$G(S_{it}^k)$	The channel efficiency on the k -th trajectory of AP i at time t .
v_i	The weight coefficient of auxiliary factor for the reward of AP i .
∇_{θ_i}	The partial derivatives of the neural network parameters θ_i .
T_q	The waiting delay for the channel buffer delay.
T_p	The delay for CSMA to complete frame transmission.
T_s	The delay of a successful frame transmission.
T_f	The delay of a failed frame transmission.
T_b	The frame transmission delay.
m_t	The gradient momentum at time t .
n_t	The exponential moving average of the squared gradient at time t .
ε	Which is used to avoid division by zero.
β	The step size of the ADAM optimization.

I. INTRODUCTION

IN RECENT decades, wireless communication networks have found success in various applications, extending from homes to industries and businesses [1], [2], [3], [4]. However, the networks' coverage is inherently limited by the propagation distance of the wireless signal. To mitigate this limitation, Mesh Wireless Networks (MWNs), also known as wireless networks in ad-hoc mode, have been developed [5], [6]. In an MWN, if an Access Point (AP) is far from a gateway, other APs situated between them can relay frames via multi-hop routing. Nonetheless, the addition of more hops adversely affects the performance of end-to-end (E2E) delay. While enhancing the transmission power of each AP results in a topology with fewer hops and thus reduces the E2E delay, it simultaneously increases overlapping and channel access collision among neighboring APs, thereby diminishing the network's channel efficiency [7], [8]. Therefore, the topology of MWNs plays a pivotal role as it directly influences network performance metrics. Effectively addressing these challenges can guide the development of network configuration solutions, thereby improving the performance of MWNs across various application domains.

There are three types of strategies for optimizing the performance of MWNs by adjusting their topology. The first type of strategy employs heuristic optimization methods based on empirical rules to optimize the topology [9], [10], [11], [12]. For instance, a study presented in [9] proposes a topology optimization methodology for communication networks that considers application characteristics, aiming to minimize network delay through the use of a Genetic Algorithm (GA) to obtain the optimal network topology. Another research [10] proposes a Topology Optimization Scheme with Global-small world (TOSG) based on ant colony optimization for the industrial internet of things. Additionally, the study [11] utilizes two particle swarm optimization algorithms— one based on a regular network structure and another based on a random network structure— to optimize the network topology. The authors in [12] suggest a Trimet graph based on optimized topology, known as Trimet Graph Optimization (TGO), to design a reliable, secure, simple, and cost-effective topology solution. Despite being simple and practical, the first type of strategy [9], [10], [11], [12] is susceptible to becoming trapped in local optima. Furthermore, heuristic optimization methods often struggle to adapt to the dynamic characteristics of MWNs, such as AP failure.

The second type of strategy, as outlined in [13], deploys Reinforcement Learning (RL) algorithms that are built upon the deep-Q-network. The intent of this approach is to adapt to dynamic features with a view to augmenting transmission throughput and diminishing delay. Given the constant information exchange between the learner and the environment [14]–[15], the RL algorithm can autonomously optimize the topology without relying on empirical rules. The exploratory action selection inherent in the RL algorithm helps to avoid getting stuck in local optima. However, a significant issue with RL algorithms is the problem of a high-dimensional action space when dealing with a large number of adjustable parameters. For instance, when N parameters are adjusted through three

actions, such as *increasing, decreasing, or maintaining transmission power*, the resulting 3^N action space leads to a substantial increase in computational demand and slows down the convergence speed during training.

To address the high-dimensional action space issue, the third type of strategy, based on distributed RL algorithms, is proposed [16], [17], [18]. The study presented in [16] applies a distributed RL method for managing swarms of Unmanned Aerial Vehicles (UAVs), where each agent is governed by individual policies, thus reducing the action space for each one. Another study presented in [17] proposes a distributed asynchronous deep RL algorithm for multi-path data transmission with the goal of enhancing system stability and throughput, and diminishing packet loss rate and delay. The multi-agent deep deterministic policy gradient (MADDPG) algorithm, as proposed in [18], allocates a critic and an actor to each agent, and allows agents to learn and plan cooperatively, which is essential in multi-agent environments. Despite the third type of strategy overcoming the high-dimensional action space issue, it still encounters convergence problems: (1) a distributed RL algorithm might become inefficient if implemented without knowledge of the global state information, and (2) the distributed RL algorithm struggles to determine the weights of multiple objectives when calculating the reward.

This paper presents a novel approach to address the challenges associated with balancing end-to-end (E2E) delay and channel efficiency in MWNs utilizing a Multi-Agent Actor-Critic Reinforcement Learning (MACRL) algorithm. The MACRL algorithm consists of a centralized critic located in the gateway and distributed actors operating in each Access Point (AP). The centralized critic plays a vital role in distributing rewards based on the global optimization status to the APs, while the distributed actors adjust the transmission power of their respective APs to minimize E2E delay and enhance channel efficiency. The main contributions are as follows:

- We employ a centralized critic to allocate rewards to each actor, addressing the problem (1) arising from the lack of global state information. Each distributed actor is trained to execute a singular action to the corresponding AP, thereby sidestepping complications associated with high-dimensional action space.
- To address the issue of weight determination for multiple objectives, as presented in problem (2), we have developed a reward balancing method. This method dynamically balances between End-to-End (E2E) delay and channel efficiency. Experimental results have validated its effectiveness.
- We've confirmed the effectiveness of the MACRL algorithm for WMNs with dynamic features, such as AP failure. The experimental results show that the proposed MACRL algorithm can dynamically optimize the network topology to adapt to environmental changes.

The remainder of this paper is organized as follows. Section II provides a description of the problem. In Section III, we introduce the MACRL algorithm. Section IV conducts numerical experiments to validate the effectiveness of the proposed MACRL algorithm. Finally, Section V draws

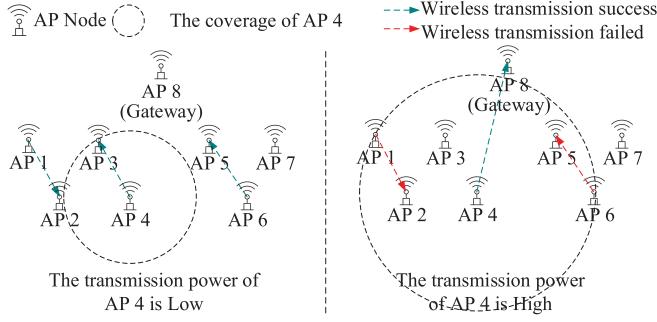


Fig. 1. Two scenarios for transmitting a frame from AP4 to the gateway. In the left, the transmission power of AP4 is low, while the right scenario depicts AP4 with high transmission power.

conclusions from the study. For the convenience of the readers, we present the nomenclature as follows.

II. PROBLEM DESCRIPTION

In this paper, we make the following assumptions:

- All APs in an MWN operate on the same channel using a fixed signal wavelength.
- The transmission power of each AP is adjustable.
- The APs are located in close proximity to each other and can form various topologies by adjusting their transmission power.
- All APs have the constant antenna gain and receive sensitivity.

The received power of AP can be determined using the Friis equation [19], as shown in (1):

$$P_{rec} = \frac{P_{trans} \times G_{trans} \times G_{rec} \times \omega^2}{(4\pi d)^2} \geq P_{min(rec_sen)} \quad (1)$$

where, P_{rec} represents the received power, P_{trans} denotes the transmission power, G_{trans} represents the gain of the transmission antenna, G_{rec} signifies the gain of the receiving antenna, ω represents the wavelength, d is the distance between the transmitting and receiving transceivers, $P_{min(rec_sen)}$ is the minimum receive sensitivity requirement.

According to (1), to meet the minimum receive sensitivity requirement, adjusting the transmission power allows for a longer communication distance, enabling the adjustment of topologies and achieving different performance levels.

To better illuminate the problem of topology optimization, an MWN with 8 nodes is present in Fig. 1 as an example. The definitions of the network's performance are as follows:

- The E2E delay refers to the sum of the delays of each hop along the path for transmitting a frame from an AP to the gateway.
- The channel efficiency represents the ratio of the average time taken for effective transmission to the average total transmission period for that frame within the channel.

When AP 4 attempts to send a frame to AP 8, the transmission power for AP 4 can have a significant impact on the network's performance. On one hand, when the transmission power of AP 4 is low, as shown on the left side of Fig. 1, the frame must be relayed by AP 3, which increases the E2E delay due to the need for multi-hop relay transmission.

However, this scenario presents an opportunity to enhance the channel efficiency by enabling simultaneous transmissions between AP 1 and AP 2, and AP 6 and AP 5. The average total transmission period decreases, improving the channel efficiency and potentially increasing the network throughput.

On the other hand, when the transmission power of AP 4 is higher, as on the right side of Fig. 1, the frame can be sent directly to AP 8, reducing the E2E delay. However, the channel's efficiency is reduced because AP 4 occupies the channel, requiring AP 1 and AP 6 to wait. This results in an increased average total transmission period within a transmission period for AP 1 and AP 6. Thus, the channel efficiency is decreased.

The relationship between network topology and performance is complex and cannot be described by a continuous function. As such, finding the optimal balance between E2E delay and channel efficiency by adjusting the transmission power levels of APs is difficult. In this paper, we model this optimization problem as a five-dimensional Markov decision process (MDP), $\langle n, \mathbb{S}, \mathbb{A}, \mathbb{R}, P \rangle$.

1) n is the number of APs. Each AP's transmission power is adjusted by a local actor, corresponding to a total of n actors.

2) $\mathbb{S}$ is the states set. The state vector $\mathbf{S}_t^k$ is defined as $\mathbf{S}_t^k = [S_{1t}^k, \dots, S_{it}^k, \dots, S_{nt}^k] \in \mathbb{S}$. Here, S_{it}^k represents the transmission power value of AP i at time t on the k -th episode trajectory.

3) $\mathbb{A}$ is the actions set and the action vector $\mathbf{a}_t^k$ is defined as $\mathbf{a}_t^k = [a_{1t}^k, \dots, a_{it}^k, \dots, a_{nt}^k] \in \mathbb{A}$. The action a_{it}^k adjusts the transmission power of AP i at time t on the k -th episode trajectory. There are three types of action: $a_{it}^k = 0$ represents increasing S_{it}^k by ΔS , $a_{it}^k = 1$ represents reducing S_{it}^k by ΔS , and $a_{it}^k = 2$ represents keeping the value of S_{it}^k unchanged. These actions can be expressed as follows:

$$S_{i(t+1)}^k = \begin{cases} S_{it}^k + \Delta S, & \text{if } a_{it}^k = 0 \\ S_{it}^k - \Delta S, & \text{if } a_{it}^k = 1 \\ S_{it}^k, & \text{if } a_{it}^k = 2 \end{cases} \quad (2)$$

where ΔS is the minimum adjustable value on transmission power.

4) $\mathbb{R}$ is the reward set. The reward vector of n APs at time t is denoted as $\mathbf{R}(\mathbf{S}_t^k) = [R(S_{1t}^k), \dots, R(S_{it}^k), \dots, R(S_{nt}^k)] \in \mathbb{R}$, where, $R(S_{it}^k)$ is the shortened form of $R(S_{it}^k | S_{i(t-1)}^k, a_{i(t-1)}^k)$, because the current state (S_{it}^k) is determined by the previous state $S_{i(t-1)}^k$ and action $a_{i(t-1)}^k$ as expressed in (2).

5) $P(S_{i(t+1)}^k | S_{it}^k, a_{it}^k)$ is the state transition matrix, which is defined as the probability of state $S_{i(t+1)}^k$ given the previous state S_{it}^k and action a_{it}^k .

For the k -th episode, the time is defined as $t = 1, 2, \dots, T$, where T is the step length of the trajectory τ_k . Then a trajectory τ_k of this episode is expressed as:

$$\tau_k = [S_1^k, \mathbf{R}(S_1^k), a_1^k, S_2^k, \mathbf{R}(S_2^k), a_2^k, \dots, S_T^k, \mathbf{R}(S_T^k), a_T^k] \quad (3)$$

The optimization problem of minimizing the E2E delay and enhancing the channel efficiency by adjusting the APs' transmission power is transferred into an MDP with the

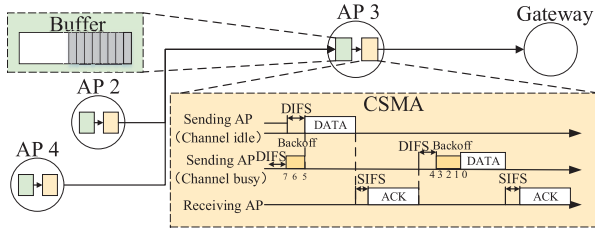


Fig. 2. MWN environment model.

objective function shown in (4).

$$\begin{aligned} \max_{\theta_1, \dots, \theta_n} \sum_{i=1}^n R_{\theta_i}^k &= \max_{\theta_1, \dots, \theta_i} \sum_{i=1}^n \sum_{t=1}^T [R(S_{it}^k) \times P_{\theta_i}(\tau_k)] \\ \text{s.t. } S_{\min} &\leq S_t^k \leq S_{\max} \end{aligned} \quad (4)$$

where, θ_i is the neural network parameters of actor i , $R_{\theta_i}^k$ is the mathematical expectation for the cumulative reward of AP i in trajectory τ_k , $\sum_{t=1}^T R(S_{it}^k)$ is the cumulative reward of AP i in trajectory τ_k , $P_{\theta_i}(\tau_k)$ is the probability of appearing trajectory τ_k under the neural network parameters θ_i of AP i , for a MDP optimization problem of RL algorithm, the probability of trajectory depends on the choice of action and the transfer of state. $S_{\min}$ and $S_{\max}$ are the minimum transmission power vector and maximum transmission power vector respectively. $S_{\max}$ is constrained by the hardware, and $S_{\min}$ limits an MWN to form a fully-connected topology.

For RL, the reward plays a crucial role in affecting learning efficiency. In particular, the transmission powers of the APs determine the topology and affect the reward function. Hence, adjusting the transmission power of the APs is crucial to achieve optimal performance. For objective function (4), how to adjust the transmission power of APs through RL to optimize the performance (reducing the E2E delay and enhancing channel efficiency) has the following problems:

- 1) Determining $R(S_{it}^k)$ to balance the optimization objective between E2E delay and channel efficiency.
- 2) Overcoming the computational challenges resulting from the high-dimensional action space while maintaining convergence speed.

We address these two problems in Section III and Section IV respectively.

III. MULTI-OBJECTIVE OPTIMIZATION MODEL

In this section, we are motivated to solve the problem of the setting and balance of reward in multi-objective optimization. First, we propose the E2E delay and channel efficiency model. Then, we present a reward balancing method to dynamically balance the MWNs' performances between the E2E delay and the channel efficiency.

A. E2E Delay Model and Channel Efficiency Optimization Model

We take AP 2, 4 through AP 3 to gateway 8 (in Fig. 1) as an example to establish an E2E delay and channel efficiency model. As shown in Fig. 2, when AP i has the state S_{it}^k , we define the frame transmission delay from AP i to the next AP as $T_p(S_{it}^k)$ and set the maximum allowable backoff stage

number as 4. According to the [20], [21], [22], and [23], the average frame transmission delay $T_p(S_{it}^k)$ can be calculated as follows:

$$T_p(S_{it}^k) = \sum_{i=0}^3 (P_f)^i (1-P_f) [iT_f(S_{it}^k) + T_s(S_{it}^k)] + 4(P_f)^4 T_f(S_{it}^k) \quad (5)$$

where $P_f = (1 - \zeta)^{Ng}$ is the probability of frame transmission failure, ζ is the probability of the AP trying to send frames, Ng is the number of neighboring APs of the current AP, $T_s(S_{it}^k)$ is the delay of successful frame transmission from AP i to the next AP with the state S_{it}^k , and $T_f(S_{it}^k)$ is the delay of failed frame transmission from AP i to the next AP with the state S_{it}^k . $T_s(S_{it}^k)$ and $T_f(S_{it}^k)$ are calculated as shown in (6)-(7), respectively.

$$T_s(S_{it}^k) = T_b(S_{it}^k) + SIFS + ACK + DIFS + \delta \quad (6)$$

$$T_f(S_{it}^k) = T_b(S_{it}^k) + DIFS + \delta + T_m(S_{it}^k) \quad (7)$$

where $T_b(S_{it}^k)$ is the frame transmission delay from AP i to the next AP with the state S_{it}^k , which is determined by the frame size and the channel rate. $T_m(S_{it}^k)$ is the average backoff delay from AP i to the next AP with the state S_{it}^k . $SIFS$, $DIFS$, and ACK are constant values, respectively corresponding to the delay required for the corresponding frame transmission, and δ is the propagation delay.

We assume that the arrivals of packets into the buffer follow a Poisson process, where the arrival rate is η and the service rate is $\mu = 1/T_p(S_{it}^k)$. According to M/M/1 queue theory [24], [25], the average buffer delay in the buffer of AP i is $T_q(S_{it}^k) = \frac{\eta}{\mu(\mu - \eta)}$.

For a MWN, the E2E delay of frame transmission from AP i to the gateway can be represented as the sum of the delays of each hop along the path. The E2E delay is expressed as $T(S_{it}^k) = \sum_i T_p(S_{it}^k)$. The channel efficiency of AP i , denoted by $G(S_{it}^k)$, refers to the ratio of the time required for effective transmission of the frame to the total transmission period within a transmission period. The efficiency of the channel for AP i is defined as $G(S_{it}^k) = \sum_i T_p(S_{it}^k) / \sum_i [T_q(S_{it}^k) + T_p(S_{it}^k)]$.

B. Reward Setting and Their Balance

To balance the E2E delay and the channel efficiency by adjusting the APs' transmission power, for AP i , we denote the E2E delay $T(S_{it}^k)$ as the main factor of the reward, and take the channel efficiency $G(S_{it}^k)$ as the auxiliary factor of the reward. Thus, the comprehensive reward of the main factor and auxiliary factor can be expressed as:

$$R(S_{it}^k) = f_1(T(S_{it}^k)) + v_i \times f_2(G(S_{it}^k)) \quad (8)$$

where, v_i is the weight coefficient corresponding to the auxiliary factor of the reward, $f_1(\cdot)$ is the function expression of the main factor of the reward, and $f_2(\cdot)$ is the function expression of the auxiliary factor of the reward.

Finding the optimal weight coefficient v_i for (8) is a challenging task. To address this problem, we need to identify the most advantageous trajectories during training for calculating the best v_i . When two trajectories, τ_p and τ_q are

considered, $p, q = 1, 2, \dots, N$ as defined in (3). When the cumulative reward of trajectory τ_p is better than the trajectory τ_q , we define that the trajectory τ_p absolutely dominates τ_q as expressed in (9). Meanwhile, the weight coefficient v_i is adjusted to prioritize the significance of E2E delay while ensuring that the auxiliary factor, channel efficiency, is not disproportionately underrepresented. Thus, we limit the impact of the main factor on the reward, ensuring it is more than ten times the influence of the auxiliary factor. This can be denoted as $f_1(T(S_{it}^k)) > 10v_i f_2(G(S_{it}^k))$, which is equivalent to equation (10). Consequently, the two conditions of calculating v_i are as follows:

1) τ_p absolutely dominates τ_q :

$$\sum_{t=1}^T R(S_{it}^p) > \sum_{t=1}^T R(S_{it}^q) \quad (9)$$

2) Limitations of v_i :

$$v_i < 0.1 \times \frac{f_1(T(S_{it}^k))}{f_2(G(S_{it}^k))} \quad (10)$$

The rationality of the conditions can be explained as follows: Condition 1) guarantees the selection of optimal trajectories with better cumulative reward for calculating v_i , which improves the training efficiency. Condition 2) provides the range of values for v_i , ensuring the proportion of the main factor and the auxiliary factor of the reward.

The optimization objectives, E2E delay and channel efficiency, are treated as the main factor and auxiliary factor in the reward. Since the impact of the auxiliary factor on the reward is relatively small, the calculation method for the reward corresponding to channel efficiency is fixed, defined as $f_2(G(S_{it}^k)) = \frac{G(S_{it}^k)}{\sum_{i=1}^n G(S_{it}^k)}$. This represents the proportion of node i 's channel efficiency value to the summation of all nodes' channel efficiency values. However, due to the significant impact of the main factor on the reward, its selection requires careful consideration. In this paper, we propose three different selections. It is important to note that in RL, the optimization of E2E delay aims to minimize the delay while maximizing the reward. Therefore, all three selections utilize the reciprocal of some form of performance measure corresponding to the E2E delay to calculate the reward.

Case 1: Let the average E2E delay of n APs serve as the performance measure, which can be represented as $f_1^{-1}(G(S_{it}^k)) = \frac{\sum_{i=1}^n T(S_{it}^k)}{n}$. The reward distribution, based on the average end-to-end (E2E) delay and the channel efficiency, is shown in (11).

$$R(S_{it}^k) = \frac{n}{\sum_{i=1}^n T(S_{it}^k)} + v_i \times \frac{G(S_{it}^k)}{\sum_{i=1}^n G(S_{it}^k)} \quad (11)$$

Case 2: Let the ratio of the delay of each AP to the summation delay of all APs serve as the performance measure, which can be represented as $f_1^{-1}(G(S_{it}^k)) = \frac{T(S_{it}^k)}{\sum_{i=1}^n T(S_{it}^k)}$. Then, the reward distribution is shown in (12).

$$R(S_{it}^k) = \frac{\sum_{i=1}^n T(S_{it}^k)}{T(S_{it}^k)} + v_i \times \frac{G(S_{it}^k)}{\sum_{i=1}^n G(S_{it}^k)} \quad (12)$$

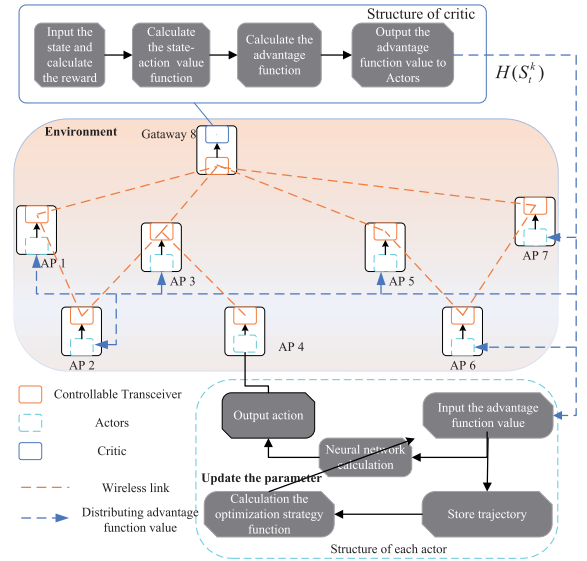


Fig. 3. A centralized-critic-distributed-actors scheme for the MWNs topology optimization.

Case 3: The performance measure in this case is defined as the ratio of the delay of the AP node to its transmission power (represented as S_{it}^k), denoted as $f_1^{-1}(G(S_{it}^k)) = \frac{T(S_{it}^k)}{S_{it}^k}$. Then, the reward distribution is shown in (13).

$$R(S_{it}^k) = \frac{S_{it}^k}{T(S_{it}^k)} + v_i \times \frac{G(S_{it}^k)}{\sum_{i=1}^n G(S_{it}^k)} \quad (13)$$

The three cases mentioned above are designed from different perspectives. In the experimental section of this paper, we will evaluate and compare the performance of three cases.

IV. MULTI-AGENT ACTOR-CRITIC REINFORCEMENT LEARNING ALGORITHM

To address the challenge posed by the high-dimensional action space and improve the efficiency of multi-agent learning efficiency, we propose a solution based on the centralized-critic-distributed-actors scheme in the MACRL algorithm. Additionally, we introduce a neural network architecture that is specifically designed for the MARL algorithm. Finally, we provide the pseudocode of the MACRL algorithm as a summary.

A. Centralized-Critic-Distributed-Actors Scheme

To solve the optimization problem of (4), we propose a MACRL algorithm with a centralized-critic-distributed-actor scheme as shown in Fig. 3.

As shown in Fig. 3, the centralized critic adopts a value update algorithm for single-step updating. Traditional distributed RL algorithms often suffer from local optimization as the agents can only perceive partial information [26]. In this paper, the centralized critic obtains the global information, calculates the rewards based on the global information, and then distributes the advantage function value to actors. As a result, the advantage function value used in actor for optimization is calculated based on global information, which avoids the problem of falling into local optimization. The distributed actors respectively select the appropriate actions

for themselves only, which overcomes the mass computation problems caused by the high-dimensional space. During the learning process, distributed actors update the parameters of the neural network and optimize APs' transmission powers to maximize the accumulated rewards.

In the above process, the critic assigns the value of the advantage function value $H(S_{it}^k)$ to each actor via the MWN. Each value of $H(S_{it}^k)$ is of type float and occupies 4 bytes. Although this process may introduce some network delay, it is important to note that this delay is inherent to the RL training process and is not directly related to the overall delay of optimizing frame transmission. As a result, we did not consider this delay in the assignment process. Additionally, the critic summarizes the information and distributes it in the form of an advantage function value, which greatly reduces the size of the frame to be transmitted, so this part of the down-link bandwidth can be ignored.

The process of solving the optimization problem of (4) with the RL algorithm can be described as N episodes of discrete Markov decision trajectories. The duration time of each episode is T . Taking the k -th episode as an example, First, APs get the state vector S_t^k , and calculate the number of neighboring APs. APs and gateway form an MWN topology based on their states (transmission powers). Next, all APs transmit a frame to the gateway, and then the critic computes the reward vector $R(S_t^k)$ by (11), (12), or (13).

Afterward, the critic calculates the state-action value function $Q(S_t^k, a_t^k)$ using (14), (15), and the iterative process is calculated by (16).

$$Q(S_t^k, a_t^k) = R(S_t^k) + \lambda \sum_{S_{t+1}^k \in \mathbb{S}} [P(S_{t+1}^k | S_t^k, a_t^k) \cdot V(S_{t+1}^k)] \quad (14)$$

$$P(S_{t+1}^k | S_t^k, a_t^k) = \prod_{i=1}^n P(S_{i(t+1)}^k | S_{it}^k, a_{it}^k = J), \quad J \in 0, 1, 2 \quad (15)$$

$$Q(S_{t+1}^k, a_{t+1}^k) = Q(S_t^k, a_t^k) + \beta [R(S_t^k) + \lambda Q(S_{t+1}^k, a_{t+1}^k) - Q(S_t^k, a_t^k)] \quad (16)$$

where, the state value function $V(S_t^k) = \sum_{t'=t}^T \lambda^{t'-t} R(S_{t'}^k)$, $V(S_t^k) = [V(S_{1t}^k), V(S_{2t}^k), \dots, V(S_{it}^k), \dots, V(S_{nt}^k)]$, λ is the discount coefficient, which determines the relative importance of rewards in the immediate future compared to rewards in the distant future for the value function, $Q(S_t^k, a_t^k) = [Q(S_{1t}^k, a_{1t}^k), Q(S_{2t}^k, a_{2t}^k), \dots, Q(S_{it}^k, a_{it}^k), \dots, Q(S_{nt}^k, a_{nt}^k)]$. For each action, $P(S_{i(t+1)}^k | S_{it}^k, a_{it}^k = J) = P(a_{it}^k = J)$, $J \in 0, 1, 2$, where, $P(a_{it}^k = J)$ is calculated in (26).

As proven in [27], the iteration process of the MDP with the state-action value function calculated by equation (16) converges when $\beta < 1$. According to (14) and (16), the critic calculates the advantage function value $H(S_t^k)$ as follows:

$$H(S_t^k) = Q(S_t^k, a_t^k) - V(S_t^k) \quad (17)$$

where $H(S_t^k) = [H(S_{1t}^k), H(S_{2t}^k), \dots, H(S_{it}^k), \dots, H(S_{nt}^k)]$.

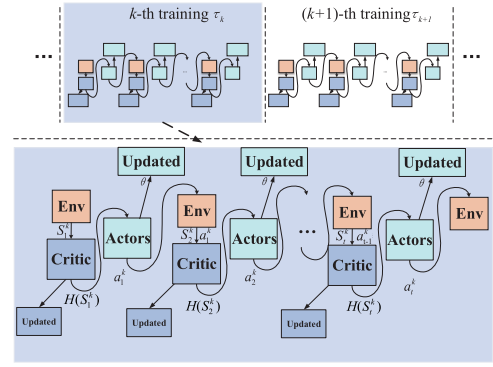


Fig. 4. MACRL training trajectory.

Then, the critic distributes the advantage function $H(S_t^k)$ and the state S_t^k to each actor. Actor i uses the advantage function $H(S_t^k)$ and S_t^k to calculate the action a_t^k by the neural network (which is presented in Section IV-B). Lastly, actor i adjusts the state $S_{i(t+1)}^k$ of AP i by (2), and then update the neural network parameters θ_i according to (21). The update of the actor's value function is achieved through a neural network architecture, which does not have a specific expression for approximation but rather through the update of neural network parameters. Therefore, it is a nonlinear parametric approximation process [28].

By looping the above optimization process T times, the training trajectory is as shown in Fig. 4. Therefore, we can get the k -th episode trajectory τ_k :

$$\tau_k = [S_1^k, H(S_1^k), a_1^k, S_2^k, H(S_2^k), a_2^k, \dots, S_T^k, H(S_T^k), a_T^k] \quad (18)$$

From (18), the critic use the advantage function $H(S_t^k)$ to replace the reward $R(S_t^k)$ in (3). Therefore, we update the neural network parameters θ_i to maximize the mathematical expectation for cumulative advantage function value $H(S_{it}^k)$ representing the objectives (E2E delay and the channel efficiency). The mathematical expectation for cumulative reward $R_{\theta_i}^k$ of AP i based on the MACRL algorithm is defined by:

$$R_{\theta_i}^k = \left(\sum_{t=1}^T H(S_{it}^k) \right) P_{\theta_i}(\tau_k) = \sum_{t=1}^T (H(S_{it}^k)) \cdot \prod_{t=1}^T [\pi_k(a_{it}^k | S_{it}^k, \theta_i) P(S_{i(t+1)}^k | S_{it}^k, a_{it}^k)] \quad (19)$$

where, $\pi_k(a_{it}^k | S_{it}^k, \theta_i)$ is the probability of neural network selecting action vector a_{it}^k in state vector S_{it}^k under the neural network parameters θ_i of AP i .

According to the policy gradient algorithm, to maximize the mathematical expectation for cumulative reward $R_{\theta_i}^k$ of all actors, it's necessary to calculate the partial derivatives of the parameters set θ_i of $R_{\theta_i}^k$. The optimization strategy function $\nabla_{\theta_i}(R_{\theta_i}^k)$ expressed as (20) can be obtained from (19):

$$\begin{aligned} \nabla_{\theta_i}(R_{\theta_i}^k) &= \nabla_{\theta_i} \left(\sum_{t=1}^T H(S_{it}^k) \right) \cdot P_{\theta_i}(\tau_k) \end{aligned}$$

$$\begin{aligned}
&= \left(\sum_{t=1}^T H(S_{it}^k) \right) \cdot P_{\theta_i}(\tau_k) \cdot \frac{\nabla_{\theta_i} P_{\theta_i}(\tau_k)}{P_{\theta_i}(\tau_k)} \\
&= \left(\sum_{t=1}^T H(S_{it}^k) \right) \cdot P_{\theta_i}(\tau_k) \cdot \nabla_{\theta_i} \log P_{\theta_i}(\tau_k) \\
&\approx \left(\sum_{t=1}^T H(S_{it}^k) \right) \\
&\quad \cdot \frac{1}{N} \cdot \frac{\partial (\log [\prod_{t=1}^T [\pi_k(a_{it}^k | S_{it}^k, \theta_i) P(S_{i(t+1)}^k | S_{it}^k, a_{it}^k)])]}{\partial \theta_i} \\
&= \frac{1}{N} \left(\sum_{t=1}^T H(S_{it}^k) \right) \\
&\quad \cdot \frac{\partial (\sum_{t=1}^T \log \pi_k(a_{it}^k | S_{it}^k, \theta_i) + \sum_{t=1}^T \log P(S_{i(t+1)}^k | S_{it}^k, a_{it}^k))}{\partial \theta_i} \\
&= \frac{1}{N} \left(\sum_{t=1}^T H(S_{it}^k) \right) \cdot \nabla_{\theta_i} \sum_{t=1}^T \log \pi_k(a_{it}^k | S_{it}^k, \theta_i) \quad (20)
\end{aligned}$$

The neural network parameter θ_i is updated as follows:

$$\theta_i \leftarrow \theta_i + \alpha \cdot \nabla_{\theta_i} (R_{\theta_i}^k) \quad (21)$$

where α is the learning rate of the actor.

B. Neural Network

In the MACRL algorithm, we design a neural network for each actor, which consists of 4 layers: The 0-th layer is the input layer (1 neuron), of which the input is the state S_{it}^k , the 1-st layer is the hidden layer (20 neurons), while the 2-nd layer is the hidden layer (20 neurons), and the 3-rd layer is the output layer (3 neurons), of which the output is the probability of three types of actions. Let $P(a_{it}^k = 0)$ denote the probability of action $a_{it}^k = 0$, $P(a_{it}^k = 1)$ represent the probability of action $a_{it}^k = 1$, and $P(a_{it}^k = 2)$ is the probability of action $a_{it}^k = 2$. Each AP randomly takes the action a_{it}^k based on the probability. Lastly, each actor uses the advantage value function to update the neural parameters.

See Fig. 5 for the neural network of actor i . In the figure, $W_{m,n}$ represents the connection weight of the m -th neuron in the input layer to the n -th neuron of the 1-st layer, $M_{m,n}$ represents the connection weight of the m -th neuron in the second layer to the n -th neuron of the 3-rd layer, Z_1 represents the linear result of the neuron in the 1-st layer, and A represents the activation function output of the neuron in the 1-st layer, Z_2 represents the linear result of the neuron in the 3-rd layer, and y represents the neuron in the 3-rd layer activation function output. The parameter of the neural network is $\theta_i = [W_{m,n}; M_{m,n}]$.

The linear output result of the first layer is obtained as follows:

$$Z_1 = W S_{it}^k \quad (22)$$

The activation function of the first layer is the ReLU function, which can be obtained as follows:

$$\text{ReLU}(z) = \begin{cases} z, & z > 0 \\ 0, & z \leq 0 \end{cases} \quad (23)$$

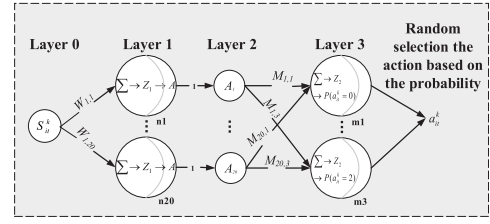


Fig. 5. The neural network of actor i .

Therefore, according to the (22)-(23), the output of first layer is obtained as follows:

$$A = \text{ReLU}(Z_1) \quad (24)$$

The linear output result of the third layer is obtained as follows:

$$Z_2 = \begin{pmatrix} M_{1,1} & \dots & M_{20,1} \\ M_{1,2} & \dots & M_{20,2} \\ M_{1,3} & \dots & M_{20,3} \end{pmatrix} A \quad (25)$$

The activation function of the third layer is the softmax function, which is derived as follows:

$$P(a_{it}^k = J) = e^{z_J} / (\sum_{j=0}^2 e^{z_j}) \quad \text{where } J = 0, 1, 2 \quad (26)$$

Therefore, according to the (25)-(26), the output of third layer is as obtained follows:

$$a_{it}^k = \text{random}(P(a_{it}^k = 0), P(a_{it}^k = 1), P(a_{it}^k = 2)) \quad (27)$$

where, $\text{random}(\cdot)$ means random selection of action a_{it}^k based on probability.

The update process of the Adam optimizer involves computing the gradient $g_t = -\nabla_{\theta_i} (R_{\theta_i(t-1)}^k)$ at time step t and taking into account the momentum from previous time steps. The gradient momentum at time step t is computed as follows:

$$m_t = \lambda m_{t-1} + (1 - \lambda) g_t \quad (28)$$

Then, the exponential moving average of the squared gradient at time step t is calculated as follows:

$$n_t = \gamma n_{t-1} + (1 - \gamma) g_t^2 \quad (29)$$

where n_0 is initialized to 0, γ represents the exponential decay rate. The default value for γ is 0.99.

To reduce the impact of bias during the initial stages of training, we perform bias correction on both the mean gradient m_t and the exponential moving average of the squared gradient n_t , which as follows:

$$\hat{m}_t = m_t / (1 - \lambda) \quad (30)$$

$$\hat{n}_t = n_t / (1 - \gamma) \quad (31)$$

The final update for the optimizer parameters θ_t are shown below:

$$\theta_t = \theta_{t-1} - \alpha * \hat{m}_t / (\sqrt{\hat{n}_t} + \varepsilon) \quad (32)$$

where ε is used to avoid division by zero and has a default value of 10^{-8} .

C. MACRL Algorithm Summary

We summarize the above process and get the MACRL algorithm as shown in Algorithm 1.

Algorithm 1 The MACRL Algorithm.

```

Initialize environment parameters, actors, critic;
for  $k = 1; k < N; k++$  do
  Reset neural network parameters  $\theta$ , get initial state  $S_1^k$ ;
  for  $t = 1; t \leq T; t++$  do
    APs form an MWN topology;
    obtains the state  $S_{it}^k$ , the E2E delay  $T(S_{it}^k)$  and the
    channel efficiency  $G(S_{it}^k)$  of each AP;
    calculates the reward vector  $R(S_t^k)$ ;
    Critic calculates the state-action value function
     $Q(S_t^k, a_t^k)$  and the advantage function vector  $H(S_{it}^k)$ ;
    Critic assigns the advantage function value  $H(S_{it}^k)$  to
    each actor;
    Actors store training trajectory  $\tau_k$ ;
    for Actor  $i$  from 1 to  $n$  do
      Actor  $i$  calculates the optimization strategy function
       $\nabla_{\theta_i}(R_{\theta_i}^k)$ ;
      Actor  $i$  updates the parameters:  $\theta_i \leftarrow \theta_i + \alpha \cdot$ 
       $\nabla_{\theta_i}(R_{\theta_i}^k)$ 
    end for
    if done or  $t > T$  then
      break;
    end if
    Each actor executes the action vector  $a_{it}^k$  and gets the
    new state  $S_{i(t+1)}^k$ ;
  end for
end for

```

V. EXPERIMENT RESULTS

In this section, the performances of the MACRL algorithm are verified by experiments. The training environment is introduced firstly, and then the performance and the dynamic of the MACRL algorithm are verified. Finally, we compare the performance of the MACRL algorithm with the distributed RL algorithm.

A. Experiment Environment

To verify the validity and performance of the MACRL algorithm, we apply the MACRL algorithm in Section IV for optimization. We generate in a random manner a scatter plot, whose size is $5000\text{cm} \times 5000\text{cm}$, as shown in Fig. 6. There are 10 AP nodes, and node 11 is a gateway.

We set the minimum and the maximum transmission power vectors as $[80, 80, 80, 78, 85, 78, 78, 74, 80, 80]$ dbm and $[97, 97, 97, 97, 97, 97, 97, 97, 97, 97]$ dbm respectively. It is worth noting that the reason for setting various transmission powers in the vectors is to maintain a connected topology. According to the [29], we defined the parameters of the environment and the training parameters of the MACRL algorithm as shown in TABLE I.

B. Experimental Results

Through the reward setting schemes from (11), (12), (13), the three cases are compared in terms of the cumulative reward

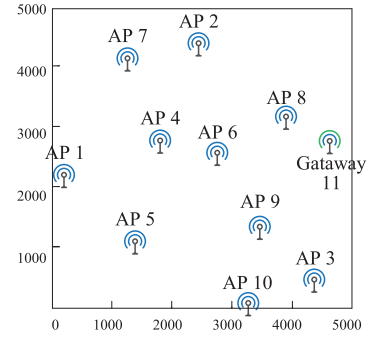


Fig. 6. Simulation environment.

Parameter name	Value	Parameter name	Value
ζ	0.01	<i>SIFS</i>	$28\mu\text{s}$
Frame size	200bits	<i>DIFS</i>	$128\mu\text{s}$
Backoff stage	4	<i>ACK</i>	$50\mu\text{s}$
Channel rate	54Mbps	δ	$1.5\mu\text{s}$
N	6000	α	0.001
n	10	γ	0.99
T	200	λ	0.9
ΔS	0.2 dBm	ε	10^{-8}

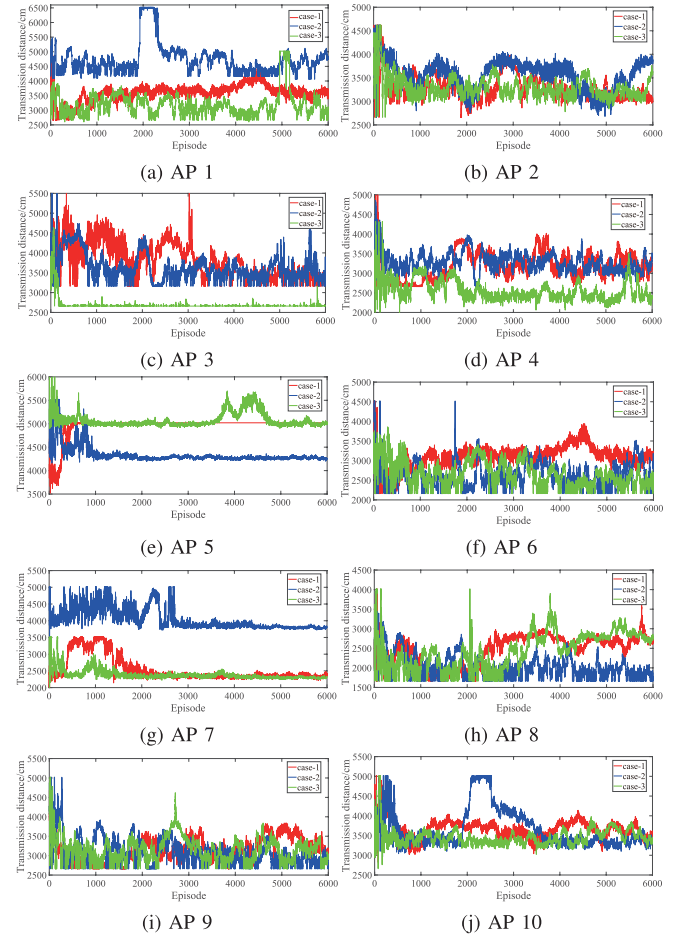


Fig. 7. The comparison in transmission distance of MACRL algorithm among case 1, case 2, case 3.

of all APs, the average E2E delay, and the transmission power of the MACRL algorithm. To better understand, we convert transmission power to the transmission distance and get the

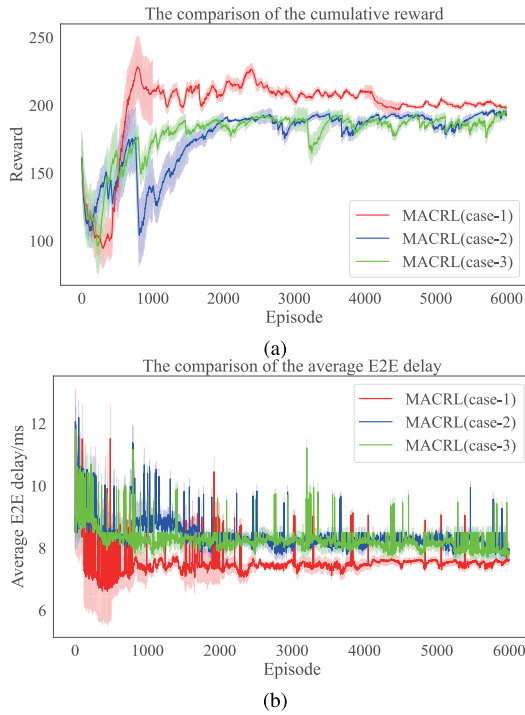


Fig. 8. The comparison of MACRL algorithm among case 1, case 2, case 3. (a) The mathematical expectation for the cumulative reward of all APs comparison. (b) Average E2E delay comparison.

comparison of the MACRL algorithm among case 1, case 2, and case 3 as shown in Fig. 7.

From Fig. 7, it is evident rewards play a crucial role in training and can significantly impact the algorithm's outcomes and convergence. The training trajectories of the three reward cases are different, leading to different delays and cumulative rewards (as shown in Fig. 8).

In Fig. 8, the shaded part is the fluctuation range during the training process. Fig. 8a illustrates the comparison of the MACRL algorithm among case 1, case 2, and case 3 in the cumulative reward of all APs. In the beginning, case 1 gets less cumulative reward of all APs than case 2 and case 3, but enjoys an improved learning efficiency after 300 training episodes, and finally stabilizes at 200. The cumulative reward of all APs in case 2 and case 3 fluctuates greatly in the initial training, increases gradually after 900 training episodes, and finally stabilizes at 190. Fig. 8b shows the comparison in average E2E delay of the MACRL algorithm among case 1, case 2, and case 3. At the beginning of training, the average E2E delay fluctuates widely. When the training episodes increase, it finally stabilizes, and the average E2E delay of them is case 1: 7.5ms, case 2: 8.1ms, case 3: 8.1ms. Therefore, case 1 performs the best in average E2E delay and stability of training.

From the experimental results of Fig. 8a and Fig. 8b, it can be found that at the beginning of the experiment, the reward of case 1 was reduced and significantly lower than that of case 2 and case 3, and the average E2E delay of case 1 was the lowest. This can be attributed to the fact that the reward allocation method of case 1 is the most appropriate, as it was able to identify a better APs' transmission power earlier than the other two cases. However, since the initial state of

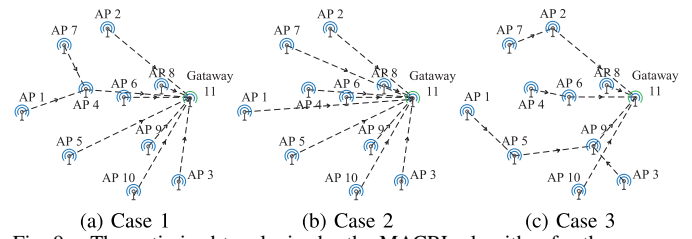


Fig. 9. The optimized topologies by the MACRL algorithm for three cases.

TABLE II

THE NUMBERS OF PARAMETERS AND FLOATING-POINT OPERATIONS FOR EACH EPISODE

actors		critic	
No. Params.	No. FLOP	No. Params.	No. FLOP
103	80	241	220

TABLE III

THE AVERAGE E2E DELAY AND THROUGHPUT COMPARISONS IN THE CASES OF THE MACRL ALGORITHM

None	Average E2E delay(ms)	Average throughput(bits/second)
Case 1	14.8212	1372.55
Case 2	12.8712	1365.8
Case 3	19.8383	1374.96

transmission power is too close to the boundaries (S_{min} or S_{max}), it led to a significant reduction in the reward once the boundary was crossed. In the subsequent optimization, such boundary-crossing results were less frequent, resulting in a rapid increase in the reward of case 1.

By utilizing the MACRL algorithm, we were able to generate optimized topologies for three cases, depicted in Fig. 9.

In all three cases, the neural networks are of equal scale, and the computational complexity remains the same. The numbers of parameters and floating-point operations for each episode are presented in TABLE II. Given the simplicity of the neural networks, the computational complexity associated with them is relatively low.

To better verify the practicality of the MACRL algorithm, the QualNet platform is used to simulate and compare the three cases in terms of average E2E delay and average throughput based on the MACRL algorithm. During the simulation, we generate the same scatter plot as Fig. 6 on the QualNet platform, and input the environment parameters and the transmission powers into the QualNet platform to obtain the MWN topology as shown in Fig. 9.

Fig. 10a shows the comparison in E2E delay of the MACRL algorithm by QualNet among case 1, case 2, case 3, and Fig. 10b shows the comparison in throughput among case 1, case 2, case 3. We calculate the average E2E delay and the average throughput as shown in Table III. The average E2E delay and average throughput are 14.8212ms and 1372.55bits/second in case 1, 11.8631ms and 1365.8bits/second in case 2, and 19.8383ms and 1374.96bits/second in case 3.

Making an overall comparison in terms of average E2E delay, the average E2E delay in case 2 is 1.95 milliseconds shorter than that in case 1, and 6.9671 milliseconds shorter than that in case 3. Among the three cases, case 2 has the lowest average E2E delay, followed by case 1. Making an

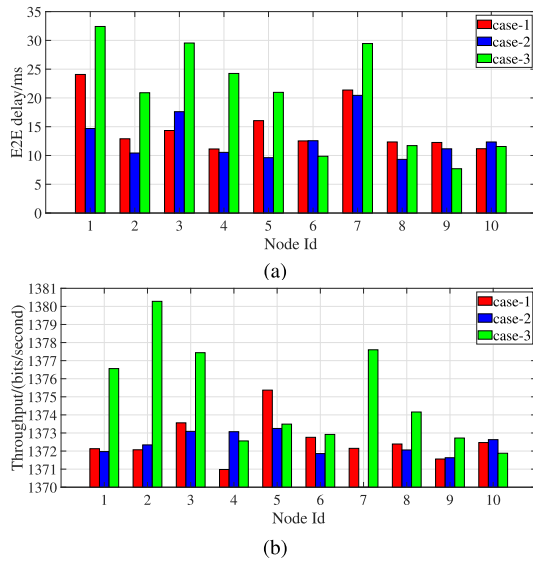


Fig. 10. The comparison of MACRL algorithm among case 1, case 2, case 3 on the QualNet. (a) E2E delay comparison. (b) Throughput comparison.

overall comparison in the average throughput, case 3 has 2.41bits/second higher average throughput than case 1, and 9.16 bits/second higher than case 1. Among the three cases, case 3 has the highest average throughput, followed by case 1. Therefore, experimental data show that case 1 performs better in terms of both considering the average E2E delay and average throughput. Case 2 performs better in the average E2E delay, but the worst in the average throughput. Case 3 gets better performance in the average throughput, but the worst performance in the average E2E delay. This implies that case 1 effectively balances the relationship between the E2E delay and the channel efficiency.

Based on the findings of this subsection, the following conclusions can be drawn: (1) The results of the cumulative reward and average E2E delay for each case demonstrate that the MACRL algorithm effectively converges and minimizes the average E2E delay, while simultaneously achieving a balance between the E2E delay and the channel efficiency. (2) Among case 1, case 2, and case 3, case 1 performs the best in terms of balancing the E2E delay and the channel efficiency. (3) The practicality of the MACRL algorithm is validated through simulations conducted on the Qualnet platform.

C. Dynamic Characteristics Experimental Results

To verify the dynamics of the MACRL algorithm, a hypothetical experiment is conducted. Assume that during training, AP 10 suddenly fails. Fig. 11a shows the cumulative reward of all APs for dynamic characteristics of the MACRL algorithm in case 1, while Fig. 11b shows the average E2E delay for dynamic characteristics of the MACRL algorithm in case 1, where the shaded part is the fluctuation range during the training process.

It can be found that when the training episodes reach 1000, AP 10 suddenly fails. Before AP 10 suddenly failed, although the rewards were increasing, the rewards and the average E2E delay fluctuated greatly. After 1000 training episodes, the cumulative rewards of all APs continued to increase after

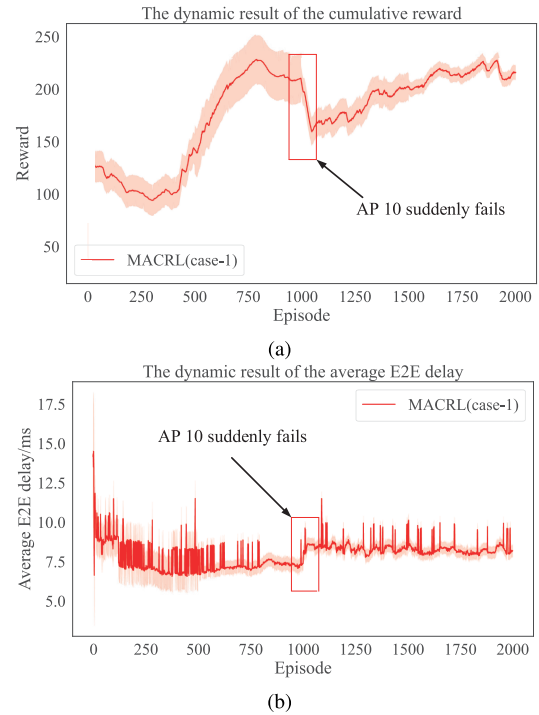


Fig. 11. The dynamic experimental results of MACRL algorithm in case 1. (a) The mathematical expectation for the cumulative reward of all APs for dynamic characteristics. (b) The average E2E delay for dynamic characteristics.

a temporary decline, and finally stabilized at 215, the average E2E delay first increased from 7ms to 9ms, then gradually decreased, and finally stabilized at 8ms.

Based on the experimental results presented in Fig. 11a and Fig. 11b, it can be observed that when an AP fails, the topology of the MWN changes abruptly, resulting in a sudden change in both reward and E2E delay. However, with continued training, the reward steadily increases and the E2E delay decreases, indicating that the topology of the MWN is dynamically optimized over time.

To better verify the dynamic characteristics of the MACRL algorithm, we conduct simulations on the QualNet platform to compare the average E2E delay and the average throughput before and after the failure of AP 10. During the simulation, Fig. 12a shows the comparison in E2E delay of the MACRL algorithm before and after AP 10 failure, while Fig. 12b shows the comparison in throughput of the MACRL algorithm before and after AP 10 failure. We calculate the average E2E delay and the average throughput. The average E2E delay and average throughput are 14.8212ms and 1372.55bits/second in the 1000 episode, 15.629ms and 1372.10bits/second in the 1100 episode, 12.9444ms and 1372.69bits/second in the 2000 episode.

The results in Fig. 12 indicate that when AP 10 fails, the average E2E delay of the MWN increases by 0.8078 milliseconds and the average throughput decreases by 0.45 bits/second from episode 1000 to episode 1100, which is consistent with the trend shown in Fig. 11a and 11b. This is due to the fact that the agent has not yet had sufficient time to optimize the transmission power, and the MWN continues to use the original topology. Consequently, the average E2E delay increases

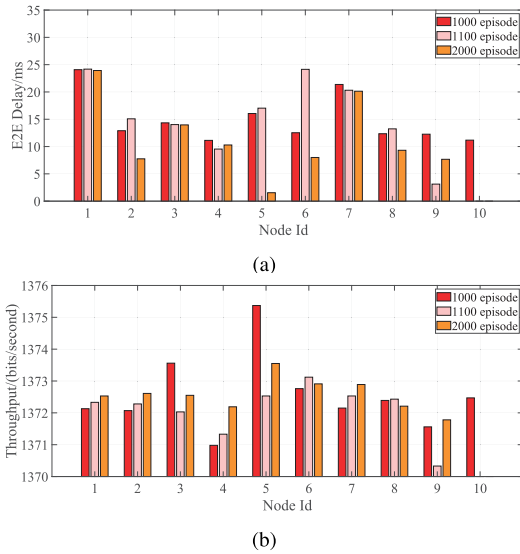


Fig. 12. The comparison of MACRL algorithm before and after AP 10 failure on the QualNet. (a) E2E delay comparison. (b) Throughput comparison.

and the channel efficiency decreases, resulting in a decrease in the average throughput. However, from episode 1100 to episode 2000, the average E2E delay of the MWN decreases by 2.6846 milliseconds and the average throughput increases by 0.59 bits/second. This phenomenon is attributed to the ability of the MACRL algorithm to rapidly adapt to changes in the MWN and respond based on the current dynamic state, achieving collaborative optimization of E2E delay and channel efficiency. Therefore, we conclude that the proposed MACRL algorithm is effective in mitigating the impact of dynamic characteristics on MWNs.

D. Comparison Experimental Results

To verify the effectiveness of the MACRL algorithm, the cumulative reward of all APs and the average E2E delay of the MACRL algorithm in case 1 is compared with the policy gradient reinforcement learning (PGRL) [30], which is a wide used distributed RL algorithm.

The PGRL is a distributed RL algorithm as well. It selects a behavior through observation information to directly carry out backpropagation, and uses reward to directly enhance and reduce the possibility of selecting behavior. It can implement a randomization strategy and is easy to calculate value function.

Fig. 13a shows the cumulative reward of all APs in comparison to this paper's algorithm with the PGRL algorithm. In the beginning, the cumulative reward of the algorithm in this paper decreased and then increased steadily, and finally stabilized at 200. However, the cumulative reward of the PGRL algorithm fluctuated little during the whole training process, rising steadily from 0 to 1000 episodes, and finally stabilizing at 180. Fig. 13b shows the average E2E delay comparison of this paper's algorithm and the PGRL algorithm. The average E2E delay of them is MACRL: 7.5ms, PGRL: 8.1ms. During the training, the average E2E delay of the PGRL algorithm is very unstable and the optimization performance is significantly worse than that of the MACRL algorithm.

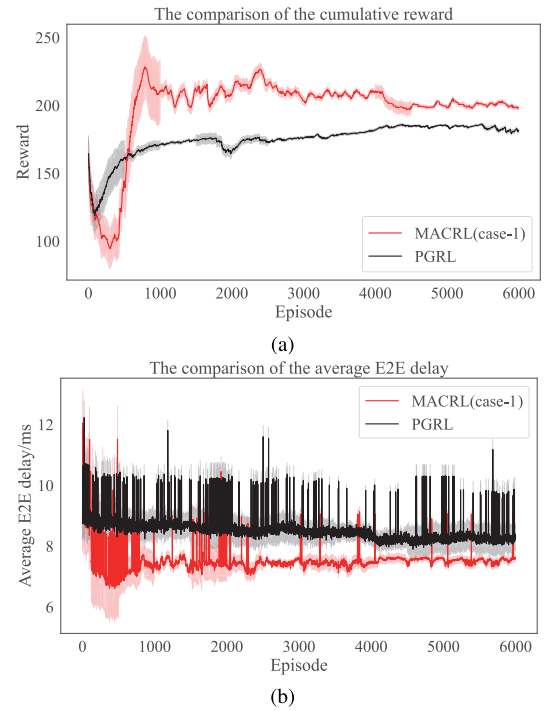


Fig. 13. The comparison of MACRL algorithm in case 1 and PGRL algorithm. (a) The mathematical expectation for a cumulative reward of all APs comparison. (b) The average E2E delay comparison.

From the experimental results of the Fig. 13a and the Fig. 13b. Although the PGRL algorithm is more stable than the algorithm of this paper in terms of reward, the stability of the E2E delay is significantly worse than the algorithm of this paper, and the final result of reward and E2E delay are also significantly worse. This is due to the disadvantage of the PGRL algorithm, which does not obtain global information and is easy to fall into local optimal solution. The algorithm of this paper has found the optimal solution at an early stage. Therefore, it can be concluded that the algorithm proposed in this paper can perform the best in balancing the E2E delay and the channel efficiency than the PGRL algorithm.

VI. CONCLUSION AND FUTURE SCOPE

In this study, we present a MACRL algorithm to minimize the average E2E delay and the channel efficiency by adjusting APs' transmission power. The problem of dynamic policy adjustments in distributed RL is solved by the use of a centralized-critic-distributed-actors scheme.

Firstly, three reward cases for the MACRL algorithm are presented and discussed. Case 1 can be integrated into a distribution system for fast E2E delay and channel efficiency optimization. Then, the MACRL algorithm is validated through dynamic simulation. It is able to satisfy the convergence and dynamic constraints while optimizing the E2E delay and the channel efficiency. Finally, the study results show better performance of the proposed MACRL algorithm in comparison with the traditional distributed RL algorithm [30]. The comparison results also illuminate that the proposed MACRL algorithm is more suitable for the optimization of the multi-objectives for distributed systems.

In the future, we plan to improve the sample efficiency of the MACRL algorithm. RL algorithms typically require a large number of samples to learn effective policies. However, in scenarios with limited data availability, such as in real-world MWN deployments, collecting a sufficient number of samples can be challenging. Hence, we aim to develop techniques that enable learning from fewer samples, such as using advanced exploration-exploitation strategies or leveraging transfer learning approaches to transfer knowledge from related environments. Furthermore, our future research will investigate the application of RL in scenarios with limited data in real-world settings. Real-world MWN deployments often exhibit dynamic and highly variable conditions that were not entirely captured in simulation environments. Therefore, we intend to explore techniques that adapt the MACRL algorithm to operate successfully in such dynamic and real-world scenarios, where the available data may be sparse or noisy. Lastly, we will focus on accelerating the convergence speed of the MACRL algorithm. By addressing these research issues, we aim to further advance the optimization capabilities of MWNs and make significant contributions to the field of wireless networking.

REFERENCES

- [1] K. W. Choi, L. Ginting, P. A. Rosyady, A. A. Aziz, and D. I. Kim, "Wireless-powered sensor networks: How to realize," *IEEE Trans. Wireless Commun.*, vol. 16, no. 1, pp. 221–234, Jan. 2017.
- [2] D. H. Lee et al., "Two-stage semi-distributed resource management for device-to-device communication in cellular networks," *IEEE Trans. Wireless Commun.*, vol. 13, no. 4, pp. 1908–1920, Apr. 2014.
- [3] Z. Liu et al., "Point cloud video streaming: Challenges and solutions," *IEEE Netw.*, vol. 35, no. 5, pp. 202–209, Sep./Oct. 2021.
- [4] W. Sun, X. Yuan, J. Wang, Q. Li, L. Chen, and D. Mu, "End-to-end data delivery reliability model for estimating and optimizing the link quality of industrial WSNs," *IEEE Trans. Autom. Sci. Eng.*, vol. 15, no. 3, pp. 1127–1137, Jul. 2017.
- [5] C. Prabha, S. Kumar, and R. Khanna, "Wireless multi-hop ad-hoc networks: A review," *IOSR J. Comput. Eng.*, vol. 16, no. 2, pp. 54–62, 2014.
- [6] Y. Xiao and Y. Pan, "Differentiation, QoS guarantee, and optimization for real-time traffic over one-hop ad hoc networks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 16, no. 6, pp. 538–549, Jun. 2005.
- [7] W. Sun, K. Wei, Z. Liu, Q. Li, and X. Xu, "Linear quadratic Gaussian control for wireless communication reliability for a mobile monitoring robot in a UHV power substation," *IEEE Syst. J.*, vol. 16, no. 3, pp. 4149–4159, Sep. 2022.
- [8] W. Sun, L. Zhang, Q. Lv, Z. Liu, W. Li, and Q. Li, "Dynamic collaborative optimization of end-to-end delay and power consumption in wireless sensor networks for smart distribution grids," *Comput. Commun.*, vol. 202, pp. 87–96, Mar. 2023.
- [9] C. Wang, N. Huang, Y. Bai, and S. Zhang, "A method of network topology optimization design considering application process characteristic," *Modern Phys. Lett. B*, vol. 32, no. 7, Mar. 2018, Art. no. 1850091.
- [10] T. Qiu, B. Li, W. Qu, E. Ahmed, and X. Wang, "TOSG: A topology optimization scheme with global small world for industrial heterogeneous Internet of Things," *IEEE Trans. Ind. Informat.*, vol. 15, no. 6, pp. 3174–3184, Jun. 2019.
- [11] F. Li and J. Guo, "Topology optimization of particle swarm optimization," in *Proc. Int. Conf. Swarm Intell.* Cham, Switzerland: Springer, 2014, pp. 142–149.
- [12] S. S. Amiripalli, V. Bobba, and S. P. Potharaju, "A novel trimet graph optimization (TGO) topology for wireless networks," in *Cognitive Informatics and Soft Computing*. Cham, Switzerland: Springer, 2019, pp. 75–82.
- [13] A. Moussaid, W. Jaafar, W. Ajib, and H. Elbiaze, "Deep reinforcement learning-based data transmission for D2D communications," in *Proc. 14th Int. Conf. Wireless Mobile Comput., Netw. Commun. (WiMob)*, Oct. 2018, pp. 1–7.
- [14] Z. Marvi and B. Kiumarsi, "Safe reinforcement learning: A control barrier function optimization approach," *Int. J. Robust Nonlinear Control*, vol. 31, no. 6, pp. 1923–1940, 2021.
- [15] A. Kuhnle, J. P. Kaiser, F. Theiß, N. Stricker, and G. Lanza, "Designing an adaptive production control system using reinforcement learning," *J. Intell. Manuf.*, vol. 32, no. 3, pp. 855–876, 2021.
- [16] F. Venturini et al., "Distributed reinforcement learning for flexible and efficient UAV swarm control," *IEEE Trans. Cognit. Commun. Netw.*, vol. 7, no. 3, pp. 955–969, Sep. 2021.
- [17] K. Liu, W. Quan, D. Gao, C. Yu, M. Liu, and Y. Zhang, "Distributed asynchronous learning for multipath data transmission based on P-DDQN," *China Commun.*, vol. 18, no. 8, pp. 62–74, Aug. 2021.
- [18] J. Li, T. Yu, and X. Zhang, "Coordinated load frequency control of multi-area integrated energy system using multi-agent deep reinforcement learning," *Appl. Energy*, vol. 306, Jan. 2022, Art. no. 117900.
- [19] O. Franek, "Phasor alternatives to Friis' transmission equation," *IEEE Antennas Wireless Propag. Lett.*, vol. 17, no. 1, pp. 90–93, Jan. 2018.
- [20] P. Chatzimisios, A. C. Boucouvalas, and V. Vitsas, "Performance analysis of IEEE 802.11 DCF in presence of transmission errors," in *Proc. IEEE Int. Conf. Commun.*, Jun. 2004, pp. 3854–3858.
- [21] I. Tinnirello, G. Bianchi, and Y. Xiao, "Refinements on IEEE 802.11 distributed coordination function modeling approaches," *IEEE Trans. Veh. Technol.*, vol. 59, no. 3, pp. 1055–1067, Mar. 2010.
- [22] Y. Xiao, "A simple and effective priority scheme for IEEE 802.11," *IEEE Commun. Lett.*, vol. 7, no. 2, pp. 70–72, Feb. 2003.
- [23] Y. Xiao, "Performance analysis of priority schemes for IEEE 802.11 and IEEE 802.11e wireless LANs," *IEEE Trans. Wireless Commun.*, vol. 4, no. 4, pp. 1506–1515, Jul. 2005.
- [24] A. H. Najim, H. S. Mansour, and A. H. Abbas, "Characteristic analysis of queue theory in Wi-Fi applications using OPNET 14.5 modeler," *Eastern-European J. Enterprise Technol.*, vol. 2, no. 9, pp. 35–43, Apr. 2022.
- [25] R. B. Cooper, "Queueing theory," in *Proc. ACM'81 Conf.*, 1981, pp. 119–122.
- [26] L. Buşoniu, R. Babuška, and B. De Schutter, "Multi-agent reinforcement learning: An overview," in *Innovations in Multi-Agent Systems and Applications—1*, D. Srinivasan and L. C. Jain, Berlin, Switzerland: Springer, 2010, pp. 183–2213.
- [27] F. S. Melo, "Convergence of Q-learning: A simple proof," *Inst. Syst. Robot., Tech. Rep.*, 2001, pp. 1–4.
- [28] E. Pashenkova, I. Rish, and R. Dechter, "Value iteration and policy iteration algorithms for Markov decision problem," in *Proc. AAAI Workshop Struct. Issues Planning Temporal Reasoning*, 1996, pp. 1–15.
- [29] G. Bianchi, "Performance analysis of the IEEE 802.11 distributed coordination function," *IEEE J. Sel. Areas Commun.*, vol. 18, no. 3, pp. 535–547, Mar. 2000.
- [30] S. Zhu, M. Ohsaki, K. Hayashi, and X. Guo, "Machine-specified ground structures for topology optimization of binary trusses using graph embedding policy network," *Adv. Eng. Softw.*, vol. 159, Sep. 2021, Art. no. 103032.



Wei Sun (Senior Member, IEEE) received the B.E. degree in automation, the M.S. degree in detection technology and automatic equipment, and the Ph.D. degree in electrical engineering from Hefei University of Technology, China, in 2004, 2007, and 2012, respectively.

He is currently a Professor with Hefei University of Technology. His research interests include wireless networks, networked control systems, and microgrids.



Qiushuo Lv received the B.S. degree in automation from the North China University of Technology, China, in 2020. He is currently pursuing the M.S. degree in control science and engineering with Hefei University of Technology, China. His research interests include wireless networks and machine learning.



Zhi Liu (Senior Member, IEEE) received the Ph.D. degree in informatics from the National Institute of Informatics. He is currently an Associate Professor with The University of Electro-Communications. His research interests include video network transmission and mobile edge computing.

He is an Editorial Board Member of *Wireless Networks* (Springer) and *IEEE TRANSACTIONS ON MULTIMEDIA*.



Yang Xiao (Fellow, IEEE) received the B.S. and first M.S. degrees in computational mathematics from Jilin University, Changchun, China, in 1989 and 1991, respectively, and the second M.S. and Ph.D. degrees in computer science and engineering from Wright State University, Dayton, OH, USA, in 2000 and 2001, respectively. He is a Full Professor with the Department of Computer Science, The University of Alabama, Tuscaloosa, AL, USA. He has directed over 20 doctoral dissertations and supervised over 20 M.S. theses/projects. He has published

over 300 Science Citation Index (SCI)-indexed journal articles (including over 70 *IEEE/ACM TRANSACTIONS*) and 300 Engineering Index (EI)-indexed refereed conference papers and book chapters related to these research areas. His research interests include cyber-physical systems, the Internet of Things, security, wireless networks, smart grids, and telemedicine.

He is an IET Fellow and an AAIA Fellow. He was a Voting Member of the IEEE 802.11 Working Group, from 2001 to 2004, involving the IEEE 802.11 (Wi-Fi) standardization work. He received the *IEEE TRANSACTIONS ON NETWORK SCIENCE AND ENGINEERING* Excellent Editor Award in 2022 and 2023. He serves/served as a member of technical program committee for more than 300 conferences. He served as a Guest Editor over 35 times for different international journals, including the *IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS* in 2022 and 2023, *IEEE TRANSACTIONS ON NETWORK SCIENCE AND ENGINEERING* in 2021, *IEEE TRANSACTIONS ON GREEN COMMUNICATIONS AND NETWORKING* in 2021, *IEEE NETWORK* in 2007, *IEEE WIRELESS COMMUNICATIONS* in 2006 and 2021, *IEEE Communications Standards Magazine* in 2021, and *Mobile Networks and Applications* (MONET) (ACM/Springer) in 2008. He also serves as the Editor-in-Chief for *Cyber-Physical Systems*, *International Journal of Sensor Networks*, and *International Journal of Security and Networks*. He has been serving as an Editorial Board Member or an Associate Editor for 20 international journals, including *IEEE TRANSACTIONS ON NETWORK SCIENCE AND ENGINEERING*, since 2022; *IEEE TRANSACTIONS ON CYBERNETICS*, since 2020; *IEEE TRANSACTIONS ON SYSTEMS, MAN, AND CYBERNETICS: SYSTEMS*, from 2014 to 2015; *IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY*, from 2007 to 2009; and *IEEE COMMUNICATIONS SURVEYS AND TUTORIALS*, from 2007 to 2014.



Qingwei Tang is currently pursuing the Ph.D. degree with the School of Electrical and Automation Engineering, Hefei University of Technology. His research focuses on pattern recognition and multi-agent reinforcement learning.



Qiyue Li (Senior Member, IEEE) received the B.E. degree in electronic engineering from Wuhan University, Wuhan, China, in 2003, and the Ph.D. degree in communication and information system from the University of Science and Technology of China, Hefei, Anhui, China, in 2008.

From 2008 to 2011, he did a post-doctoral research with the School of Computer Science and Technology, University of Science and Technology of China. He is currently a Professor with Hefei University of Technology, Hefei. His research interests include

wireless networks and indoor localization using wireless networks.



Daoming Mu is currently a Senior Engineer with Hefei University of Technology, Hefei, China. His research interests include wireless sensor networks and smart grids.